

상품 정보 파싱 엔진 — 구조·논리 설계 문서

3단계 폴백(fallback) 아키텍처의 구현 완료본에 대한 구조와 논리 정리. 전략 문서 「상품 정보 수집(파싱 엔진) 전략 설계」의 후속 문서이며, 발표 자료 및 개발 공유용.

대상 코드: `server/` (브랜치 `cursor/three-tier-parsing-engine-adf2`, PR #7) · 작성일: 2026-07-25 · 테스트: 오프라인 70건+ 통과 + 실사이트 라이브 검증

1. 개요 — 무엇을, 왜 이렇게 만들었나

사용자가 외부 쇼핑 앱에서 공유한 상품 URL로부터 **쇼핑몰 이름·URL·상품명·판매가(할인가)·정가·할인율·이미지**를 자동 추출해 **하나의 일관된 구조로 정규화**하는 엔진이다. 초기에 검토했던 웹뷰(헤드리스 브라우저) 렌더링 방식은 ① 이용약관의 "자동화 수단에 의한 접근 금지" 조항과 충돌 소지, ② 화면 개편 때마다 깨지는 취약성, ③ 렌더링 비용, ④ 발표 방어 불가의 네 가지 이유로 **기각**했고, 구 코드의 Playwright 의존성도 완전히 제거했다.

핵심 원칙: 엔진의 모든 단계는 **데이터 제공자가 의도한 경로**만 사용한다. 공식 API(Tier 1), 사이트가 공유·검색 노출용으로 스스로 공개한 표준 메타데이터(Tier 2), 그리고 그마저 없으면 사용자 보완(Tier 3). 다른 서비스 User-Agent 위장·JS 렌더링·로그인 세션·WebView는 코드에 없다. HTTP GET은 정직한 WishListBot/1.0 으로 URL당 최대 1회다.

2. 전체 구조

공유받은 URL

- └─ [Tier 1] 공식 오픈 API 연동 (쿠팡·네이버·11번가 — 오픈마켓형)
 - └─ 플랫폼 식별 → 상품 식별자 추출 → API 조회 → 매칭
 - └─ 성공: 상품명·이미지·판매가(+정가 가능 시) 확보, 가격 추적 가능
- └─ [Tier 2] 표준 메타데이터 파싱 (29CM·W컨셉 등 — 편집샵형)
 - └─ WishListBot HTTP GET 1회 → JSON-LD(Product) 우선 → Open Graph 보완
 - └─ 성공 조건: 상품명 + 이미지 + 판매가 (정가·할인율은 있으면 함께 저장)
- └─ [Tier 3] 미리보기 수준 + 사용자 보완 (그 외 전부 — 절대 실패하지 않음)
 - └─ 얻은 것은 전부 미리 채우고, 빠진 항목(주로 가격)만 사용자가 입력
 - └─ Tier와 무관하게 `original_url` 은 항상 저장 (탭 → 앱/웹으로 열기)

상위 단계가 실패하면 자동으로 다음 단계로 내려가며(graceful degradation), 어떤 단계로 수집됐는지(`source_type`)와 각 단계의 시도 기록(`attempts`)을 결과에 함께 남긴다.

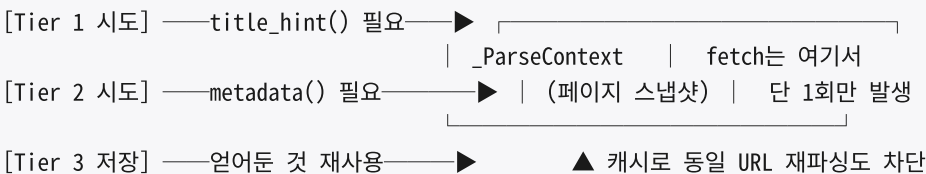
2.1 모듈 구성 — 문서의 절이 파일 하나에 대응된다

파일	역할	전략 문서 근거
engine/pipeline.py	3단계 폴백 오케스트레이터 (엔진 본체). 티어 시도·폴백·기록	5절 (폴백 아키텍처)
engine/models.py	Product · ParseResult · TierAttempt 정규화 모델	5.1 (데이터 모델)
engine/urltools.py	도메인→플랫폼 식별 + URL에서 상품 식별자 추출 (플랫폼 레지스트리)	5.2 흐름 1-2
engine/fetch.py	HTTP GET 1회 + robots.txt 존중 + 정직한 User-Agent (준수 원칙 강제 계층)	5.3 준수 원칙
engine/metadata.py	JSON-LD(Product) 우선 파싱 → Open Graph 보완, 가격 정규화	4절, 5.3
engine/tiers/tier1_api.py	네이버·쿠팡·11번가 API 어댑터 + 매칭 로직(식별자→유사도)	5.2 (Tier 1)
engine/tiers/tier2_metadata.py	메타데이터 → Product 정규화, Tier 2 성공 판정	5.3 (Tier 2)
engine/tiers/tier3_preview.py	미리보기 수준 저장 (절대 실패하지 않는 마지막 단계)	5.4 (Tier 3)
engine/cache.py	TTL 캐시 — 동일 URL 중복 요청 방지, API rate limit 대응	5.2 제약, 5.3 원칙
api_server.py	FastAPI POST /parse — 앱(Flutter)이 호출하는 얇은 HTTP 계층	—
tests/ (82건)	네트워크 없이 도는 오프라인 테스트 (폴백 순서·불변식·매칭 규칙 검증)	—

3. 설계 불변식 — 이 엔진이 항상 지키는 네 가지

불변식 1. 상품 페이지 HTTP GET은 URL당 최대 1회

한 번의 파싱 동안 모든 티어는 `_ParseContext` 객체를 공유한다. 어느 티어가 먼저 페이지를 필요로 하든 `fetch`는 한 번만 일어나고 결과가 재사용된다. 예컨대 Tier 1의 네이버 어댑터가 검색 키워드(페이지 제목)를 얻으려고 페이지를 가져왔는데 매칭에 실패하면, Tier 2는 **이미 받아둔 그 HTML**로 메타데이터를 파싱한다. "브라우저의 최초 요청 1회와 동일하다"는 발표 논리가 코드 구조로 보장된다.



불변식 2. Tier 3은 실패하지 않는다 → 저장은 항상 성공한다

파싱 엔진의 유일한 실패 경로는 "입력이 http(s) URL이 아님"(422) 뿐이다. 네트워크 오류, 403 차단, 메타데이터 부재, robots.txt 거부 — 어떤 상황에서도 원본 링크만으로 미리보기 수준의 저장이 성립하므로, **파싱 엔진의 실패가 앱의 실패**

로 이어지지 않는다.

불변식 3. 준수 원칙은 사람이 아니라 코드 계층이 강제한다

약관·윤리 방어선은 `engine/fetch.py` 한 곳에 구현되어 있어, 어떤 티어도 이를 우회해서 페이지를 가져올 수 없다.

준수 원칙 (전략 문서 5.3)	구현
robots.txt 확인·존중	<code>RobotsPolicy</code> 가 <code>fetch</code> 전에 검사. 차단 시 우회 없이 <code>FetchBlocked</code> → Tier 3 폴백. 호스트 별 24h 캐시
User-Agent에 신원 명시	<code>WishlistBot/1.0 (+저장소 URL; link-preview fetch on explicit user save)</code> — 카카오톡 <code>kakaotalk-scrap</code> 과 같은 관례
저장 시점 1회 요청, 순회 크롤링 금지	엔진에 URL 목록 순회·탐색 기능 자체가 없음. 진입점은 <code>parse(url)</code> 하나
동일 URL 중복 요청 방지	파싱 결과 TTL 캐시 (기본 6시간)
JS 렌더링·로그인·봇 차단 우회 금지	의존성에 브라우저가 없음 (Playwright 제거). 403/429는 사이트의 의사표시로 받아들이고 폴백

불변식 4. 확신 없는 매칭은 하지 않는다

Tier 1의 검색 결과 매칭에서 상품 식별자가 일치하지 않고 이름 유사도도 기준(0.55) 미달이면, 억지로 채택하지 않고 Tier 2로 폴백한다. 틀린 상품 정보를 저장하는 것보다 한 단계 낮은 확실한 정보가 낫다는 판단이며, 이 기준 자체가 테스트로 고정되어 있다.

4. 할인을 계산 — 정가 > 판매가일 때만 `discount_rate = round((정가-판매가)/정가×100)` .

5.2 성공 판정

상품명 + 이미지 + 가격을 모두 확보해야 Tier 2 성공이다. 하나라도 없으면 Tier 3으로 폴백하되, 얻은 제목·이미지는 미리 채우고 빠진 필드(`missing_fields` , 주로 가격)만 사용자가 입력한다.

안정성 근거: OG와 JSON-LD는 화면용 마크업이 아니라 표준 스펙이다. 사이트가 화면(CSS 클래스)을 개편해도 이 태그들은 유지되므로, 사이트별 셀렉터 유지보수가 **존재하지 않는다**. 범용 파서 하나가 모든 사이트를 커버하며, 이것이 "마크업 변화에 능동적으로 대응하는 파싱 엔진"에 대한 답의 절반이다 (나머지 절반은 폴백 체인).

6. Tier 3 — 미리보기 수준 저장 + 사용자 보완

카카오톡 미리보기 카드 수준의 최소 정보(제목·대표 이미지·원본 링크)를 저장하되, **얻은 정보는 전부 미리 채워두고** 빠진 항목만 `missing_fields` 로 앱에 알려준다. 앱의 저장 화면은 이 목록의 필드만 사용자에게 확인·입력받으면 된다.

- UX 원칙: "전부 수동 입력"이 아니라 "**빠진 것만 확인**" — 체감이 완전히 다르다.
- 최악의 경우(모든 것이 실패)에도 원본 링크 하나로 저장이 성립한다.

7. 데이터 모델 — 정규화와 출처 기록

```
Product {
  title, image_url,
  price,           // 판매가/할인가 (실제 결제에 가까운 가격)
  original_price,  // 정가 (할인 전). 없으면 null
  discount_rate,   // 할인율 %. 정가 > 판매가일 때만 계산
  currency, brand, category,
  seller,          // 판매자/입점 스토어 (JSON-LD offers.seller, 네이버 mallName)
  original_url,    // 원본 상품 페이지 (탭 시 앱 있으면 앱, 없으면 웹)
  source_platform, // musinsa, coupang, 29cm, ...
  platform_label,  // 화면 표시용 쇼핑몰 이름 ("무신사", 미등록물은 og:site_name)
  source_type,     // API | METADATA | MANUAL ← 어느 티어로 수집됐나
  price_trackable, // 가격 추적 가능 여부 (source_type에서 파생)
  fetched_at
}

ParseResult {
  product,          // Tier 3 덕분에 항상 존재 (original_url 항상 포함)
  resolved_tier,    // 최종 채택된 티어 (1|2|3)
  missing_fields,   // 사용자 입력이 필요한 핵심 필드 (title/image_url/price)
  attempts[],       // 각 티어의 시도 기록 {tier, name, outcome, reason}
  from_cache
}
```

수집 필드	필수(핵심)	출처 예
platform_label (쇼핑몰 이름)	△	플랫폼 레지스트리 / <code>og:site_name</code>
original_url	항상	사용자 입력/공유 텍스트
title (상품명)	핵심	API / JSON-LD name / <code>og:title</code>
price (판매가·할인가)	핵심	API <code>lprice</code> / <code>offers.price</code> / <code>product:sale_price</code>
original_price (정가)	있으면	API <code>hprice</code> / <code>listPrice</code> / <code>product:original_price</code>
discount_rate	파생	정가·판매가로 계산
image_url	핵심	API / JSON-LD image / <code>og:image</code>

티어	source_type	성공 조건	가격 변동 추적
Tier 1 (API)	API	API 매칭 성공 (판매가 포함)	○ — <code>price_trackable=true</code>
Tier 2 (메타데이터)	METADATA	제목 + 이미지 + 판매가	△ — 현재 false (재파싱 검토)
Tier 3 (수동)	MANUAL	항상 성공. 얻은 필드 미리 채움 + 빠진 칸 입력	✗

가격 변동 추적은 공식 API 플랫폼부터 지원하고 단계적으로 확대한다 — 기능 축소가 아니라 적법하고 지속가능한 범위를 스스로 정의한 엔지니어링 판단.

8. 모니터링 — 구조 변화를 스스로 감지하는 엔진

모든 파싱 결과는 `attempts` (티어별 성공/스킵/실패 + 사유)를 남기고, 파이프라인이 `platform · resolved_tier · source_type · missing` 을 로그로 집계한다.

예: 특정 플랫폼의 `resolved_tier` 분포가 2 → 3 으로 쏠리기 시작
⇒ 해당 사이트가 메타데이터 제공을 중단/변경했다는 신호
⇒ 사용자에게는 여전히 저장이 성공하고 있으므로 (Tier 3),
운영자는 장애가 아니라 "품질 저하 알림"으로 여유 있게 대응

"마크업 변화에 능동적으로 대응할 파싱 엔진 확보 방안"이라는 평가 피드백에 대해, 이 엔진은 ① 애초에 화면 마크업을 읽지 않고(Tier 2), ② 어떤 단계가 깨져도 하위 단계로 자동 전환되며(폴백), ③ 깨졌다는 사실 자체를 데이터로 감지한다(모니터링) — 세 겹으로 답한다.

9. 검증 결과 (정직한 기록)

9.1 오프라인 테스트 — 82건 전부 통과

- URL 해석: 10개 플랫폼 식별·식별자 추출, 유사 도메인 오인 방지
- 메타데이터: JSON-LD 실사이트 변형 6종, OG, 가격 표기 정규화, 깨진 블록 내성
- Tier 1: 식별자 우선 매칭, 유사도 폴백, 확신 미달 시 거부, 키 미설정 스킵, HTTP 오류 → `AdapterError`
- 파이프라인: 폴백 순서, **GET 1회 불변식**, robots 차단 → Tier 3, 캐시 적중, 단축링크 재식별, 잘못된 URL 422

9.2 실사이트 라이브 검증 (2026-07-25)

사이트	결과	내용
29CM 실상품 2건	Tier 2 성공	JSON-LD에서 상품명·이미지· 가격 ·브랜드·카테고리 전부 확보, <code>missing_fields: []</code>
무신사	WishlistBot unfurl	robots.txt는 <code>Disallow: /</code> 이지만 WishlistBot GET은 200+OG. 판매가가 메타에 없으면 Tier 3(제목·이미지 미리 채움 + 가격 입력)
에이블리	Tier 3	Cloudflare가 WishlistBot을 403 → 다른 UA 위장 없이 Tier 3 (공유 텍스트 상품명 있으면 제목 미리 채움)
API 서버	정상	정상 URL 200 + 정규화 응답 / 비정상 입력 422

준수 원칙: HTTP는 항상 `WishlistBot/1.0` 만 사용한다. Facebook/Slack 등 다른 서비스 UA를 빌려 쓰는 `compat unfurl`은 하지 않는다. robots.txt가 거부해도 사이트에 공개된 OG를 WishlistBot으로 1회 읽는 것은 “사용자 저장 링크 미리보기” 수준이며, 403이면 Tier 3으로 내려간다.

10. API 명세

POST /parse — 요청: {"url": "https://www.29cm.co.kr/products/1577769"}

```
{
  "product": {
    "title": "퀸 비키니 브라_화이트",
    "image_url": "https://img.29cm.co.kr/next-product/...jpg",
    "price": 109000,
    "currency": "KRW",
    "brand": "아그넬",
    "category": "비키니",
    "original_url": "https://www.29cm.co.kr/products/1577769",
    "source_platform": "29cm",
    "source_type": "METADATA",
    "price_trackable": false,
    "fetched_at": "2026-07-25T09:07:00Z"
  },
  "resolved_tier": 2,
  "missing_fields": [],
  "attempts": [
    {"tier": 1, "name": "open_api", "outcome": "skipped",
     "reason": "platform '29cm' does not provide an open API"},
    {"tier": 2, "name": "metadata", "outcome": "success",
     "reason": "extracted via json-ld, open-graph"}
  ]
}
```

- 응답은 항상 200 — 유일한 실패는 http(s) URL이 아닌 입력의 422.
- `missing_fields` 가 비어 있지 않으면 앱은 해당 필드만 사용자에게 확인받는다.
- 실행: `cd server && pip install -r requirements.txt && uvicorn api_server:app` → `/docs` 에서 즉시 테스트 가능.

11. 예상 질문과 답변

Q. 웹뷰 방식과 뭐가 다른가? 결국 남의 페이지를 읽는 것 아닌가?

A. 요청 성격이 다르다. 웹뷰는 페이지 전체를 렌더링(JS 실행, 수십 개 부속 요청)해 화면용 DOM을 긁지만, 이 엔진은 브라우저의 최초 요청과 동일한 **HTTP GET 1회**로, 사이트가 **기계 판독용으로 의도적으로 공개한 표준 필드**(OG/JSON-LD)만 읽는다. 카카오톡·슬랙이 링크 미리보기를 만들 때 하는 동작과 기술적으로 동일하며, 이 "1회" 제약은 코드 구조 (`_ParseContext` 공유)로 강제된다.

Q. 사이트가 크롤링을 막으면?

A. 막으면 따른다. `robots.txt`가 거부하면 우회 없이 Tier 3으로 내려가고, 403·429 응답도 사이트의 의사표시로 받아들인다. 실제로 무신사가 현재 이 경우이며, 그래도 저장은 성공한다(미리보기 + 사용자 보완). 봇 차단을 "따돌리는" 코드가 저장소에 존재하지 않는 것이 웹뷰 방식과의 결정적 차이이다.

Q. API 키가 아직 없는데 Tier 1은 어떻게 되나?

A. 어댑터가 `skipped` 로 기록되고 Tier 2로 자연 풀백한다. 키는 환경변수 주입이라 발급 즉시 코드 변경 없이 Tier 1이 켜진다. 반대로 API 장애·rate limit 시에도 같은 경로로 서비스가 유지된다.

Q. 검색 API 매칭이 엉뚱한 상품을 고르면?

A. 식별자 일치가 최우선이고, 이름 유사도는 기준(0.55) 미달이면 채택하지 않는다. 확신이 없으면 Tier 2의 확실한 정보 (그 페이지 자체의 메타데이터)로 폴백하는 쪽을 택했고, 이 동작은 테스트로 고정되어 있다.

Q. 새 쇼핑몰 지원은 얼마나 걸리나?

A. 메타데이터가 있는 사이트는 **0줄** — Tier 2 범용 파서가 그대로 커버한다. 플랫폼 식별·식별자 추출까지 원하면 `urltools.py` 레지스트리에 항목 1개 추가, 공식 API 연동까지 원하면 어댑터 클래스 1개 추가로 끝난다.